Elektro in računalniška šola, Šolski center Ptuj

Poročilo pri predmetu

VARNOST IN ZAŠČITA V INFORMATIKI

GIT HUB

Adam Školnik

Mentor: Aljaž Šešo

Ptuj, september 2024

KAZALO VSEBINE

1. Teoretični del.....3
1.1 Kaj je GitHub in kako deluje?.....3
1.2 Zakaj se uporablja GitHub?.....3
1.3 Uporaba GitHuba v različnih ekipah razvijalcev.....4
1.4 Kaj je merge v okolju GitHub in kako pripomore pri ekipnem delu?.....5
2. Praktični del in izvedeni ukazi.....6
2.1 Podrobna razlaga posameznih ukazov in njihove vloge:.....6
3. Težave.....7
4. Ugotovitve.....8
5. Zaključek.....8
6. Viri :.....9

1. TEORETIČNI DEL

1.1 Kaj je GitHub in kako deluje?

GitHub je vodilna svetovna spletna platforma za gostovanje repozitorijev, nadzor nad različicami kode ter upravljanje projektov razvoja programske opreme. Deluje na podlagi distribuiranega sistema za nadzor različic **Git**, ki ga je leta 2005 ustvaril Linus Torvalds.

Osnovno delovanje GitHuba temelji na konceptu **odjemalec-strežnik** v kombinaciji z distribuirano arhitekturo:

- **Lokalni repozitorij:** Vsak razvijalec ima na svojem lokalnem računalniku celotno kopijo projekta, vključno s celotno zgodovino vseh sprememb in vej. Lokalno izvede spremembe, jih pripravi (*staging*) ter potrdi (*commit*).
- **Oddaljeni repozitorij (Remote Repository):** Centralna kopija projekta gostuje na strežnikih platforme GitHub. Razvijalci svoje lokalno potrjene spremembe s pomočjo ukaza git push pošljejo na GitHub, spremembe drugih članov ekipe pa pridobijo z ukazom git pull ali git fetch.

Glaven element projekta na GitHubu je **repozitorij** (*repository* ali *repo*). Poleg same izvirne kode vsebuje tudi skrito mapo .git, v kateri Git hrani vse metapodatke, dnevnike sprememb (*commit log*) ter kazalce na posamezne različice datotek.

1.2 Zakaj se uporablja GitHub?

Uporaba GitHuba je postala industrijski standard v sodobnem inženirstvu programske opreme iz naslednjih ključnih razlogov:

1. **Popoln nadzor nad različicami (*Version Control*):** Vsaka sprememba je zabeležena kot unikatna veriga sprememb (*commit*), opremljena s časovnim žigom, avtorjem in opisom. To omogoča enostaven vpogled v zgodovino ter hitro povračilo (*rollback*) na katerokoli prejšnje stabilno stanje kode ob pojavu hroščev ali napak.
2. **Varno shranjevanje in varnostno kopiranje v oblaku:** Koda ni vezana zgolj na lokalno opremo posameznika. Z objavo na GitHubu je varno arhivirana na oddaljenih strežnikih, kar preprečuje izgubo dela ob okvari strojne opreme.

3. **Asinhrono in distribuirano sodelovanje:** Več sto ali celo tisoč razvijalcev lahko hkrati dela na istem projektu brez tveganja, da bi si medsebojno prepisali ali uničičili kodo, saj vsak posameznik deluje v svojem ločenem okolju ali veji.
4. **Pregled in ocena kode (*Code Review & Pull Requests*):** Preden se nova funkcionalnost združi z glavno vejo projekta, sistem omogoča podroben pregled kode s strani izkušenejših sodelavcev, dodajanje komentarjev, predlogov za izboljšave in avtomatsko testiranje.
5. **Upravljanje projektov in sledenje hroščem (*Issue Tracking & Projects*):** GitHub vsebuje vgrajena orodja za vodenje projektov (Kanban table), sledenje opozorilom (*Issues*), označevanje prioritet in dodeljevanje nalog posameznim članom ekipe.

1.3 Uporaba GitHuba v različnih ekipah razvijalcev

Način uporabe GitHuba se prilagaja specifikam posamezne veje razvoja programske opreme:

1. Ekipa za razvoj spletnih aplikacij (*Web Development*):

- **Lokalno in testno okolje:** Spletni razvijalci pogosto uporabljajo model vej *Git Flow* (npr. veje main, develop, feature/*, hotfix/*).
- **CI/CD Integracija (Neprekinjena integracija in nameščanje):** GitHub povezujejo z orodji, kot so GitHub Actions, Vercel ali Netlify. Takoj ko razvijalec izvede merge v glavno vejo main, avtomatiziran sistem samodejno požene teste, zgradi aplikacijo in jo brez prekinitve delovanja objavi na produkcijski strežnik.

2. Ekipa za razvoj računalniških iger (*Game Dev*):

- **Upravljanje velikih datotek (Git LFS):** Projektne mape iger vsebujejo obsežne binarne datoteke (3D modele, teksture visoke ločljivosti, zvočne posnetke in animacije). Standardni Git za take datoteke ni optimiziran, zato ekipe uporabljajo razširitev **Git LFS** (*Large File Storage*), ki binarne datoteke hrani na posebnih strežnikih, v Git repozitoriju pa pusti le majhne kazalce.
- **Ločevanje vlog:** Omogoča nemoteno vzporedno delo programerjem logike (ki spreminjajo kodo C#, C++ ali Python) in grafičnim oblikovalcem ter oblikovalcem stopenj (*level designers*).

3. Ekipa za razvoj namiznih aplikacij (*Desktop Dev*):

- **Večplatformno testiranje:** Razvijalci namiznih aplikacij (za Windows, macOS in Linux) uporabljajo GitHub Actions za samodejno prevajanje (*compilation*) ter testiranje kode na različnih operacijskih sistemih ob vsakem push ukazu.

- **Upravljanje izdaj (*Releases*):** GitHub omogoča enostavno objavo končnih izvršljivih datotek (.exe, .dmg, .AppImage) skupaj z opombami ob izdaji (*release notes*) in verzioniranjem po standardu Semantic Versioning (npr. v1.0.0).

1.4 Kaj je merge v okolju GitHub in kako pripomore pri ekipnem delu?

Merge (združevanje) je eden najpomembnejših postopkov v sistemu Git in okolju GitHub. Gre za proces, pri katerem se zgodovina in spremembe iz ene veje (npr. delovne veje feature-prijava) vključijo in združijo v drugo vejo (običajno glavno vejo main ali master).

Glavni vidiki delovanja in prispevek k ekipnemu delu:

1. **Izolacija dela in stabilnost glavno veje:** Razvijalci nikoli ne delajo neposredno na glavni veji main, ki predstavlja stabilno, delujočo različico aplikacije. Namesto tega ustvarijo svojo vejo (*feature branch*), kjer lahko prosto eksperimentirajo in razvijajo kodo. Šele ko je koda v celoti dokončana in testirana, se z ukazom merge prenese v glavno vejo.
2. **Zahteve za združitev (*Pull Requests - PR*):** V okolju GitHub se merge običajno izvede preko zahteve za združitev (*Pull Request*). PR služi kot forum, kjer avtor predstavi svoje spremembe, ostali člani ekipe pa kodo pregledajo, razpravljajo o rešitvah in po potrditvi odobrijo združitev.
3. **Reševanje konfliktov združevanja (*Merge Conflicts*):** Če dva razvijalca hkrati spremenita isto vrstico v isti datoteki na različnih vejah, Git ob poskusu izvedbe merge ne more samodejno odločiti, katera različica je pravilna. V tem primeru ustvari konflikt (*merge conflict*). Git označi sporne vrstice v datoteki, razvijalec pa mora konflikt ročno razrešiti, preveriti pravilnost in šele nato dokončati združevanje.

2. PRAKTIČNI DEL IN IZVEDENI UKAZI

Za potrebe vaje je bil v sklopu predmeta ustvarjen nov oddaljeni repozitorij na spletni platformi GitHub z natančnim imenom **"VZI praksa"**.

Nalaganje materialov in datotek je potekalo preko ukazne vrstice (Terminal / CMD) v dveh zaporednih fazah.

2.1 Podrobna razlaga posameznih ukazov in njihove vloge:

1. cd mapaVZI

- **Pomen:** Ukaz cd (Change Directory) se uporablja za premikanje po imenikih v ukazni vrstici.
- **Vloga:** Z njim vstopimo v mapo, kjer imamo shranjene datoteke predmeta (poročila, datoteke vaje), ki jih želimo poslati na GitHub.

2. git init

- **Pomen:** Inicializira nov prazen Git repozitorij v trenutni mapi.
- **Vloga:** V mapi se ustvari skrita podmapa .git, ki služi kot lokalna podatkovna baza. Od tega trenutka naprej Git v tej mapi spremlja vse spremembe, ustvarjanje in brisanje datotek.

3. git add .

- **Pomen:** Ukaz doda vse datoteke iz delovne mape (Working Directory) v področje za pripravo (Staging Area ali Index).
- **Vloga:** Pika (.) pomeni, da vključimo vse spremenjene, ustvarjene ali izbrisane datoteke v mapi. S tem Gitu poveste, katere spremembe naj pripravi za naslednji "posnetek" stanj (commit).

4. git commit -m "Sporočilo commit-a"

- **Pomen:** Ustvari trajni posnetek stanja pripravljenih datotek v lokalni zgodovini Git-a.
- **Vloga:** Z zastavico -m dodamo obvezno sporočilo, ki na kratko in jasno opisuje, kaj smo v tem koraku spremenili. V prvem koraku smo uporabili sporočilo "Prvi commit projekta - vsa dotedanja poročila in prazno poročilo Vaje 2", v drugem pa "Dopolnjeno poročilo za Vajo 2 s teorijo in ugotovitvami".

5. git remote add origin POVEZAVA_DO_GIT_REPOZITORIJA

- **Pomen:** Poveže lokalni Git repozitorij z oddaljenim repozitorijem na spletni platformi GitHub.
- **Vloga:** Ime origin je standardno ime (ali alias) za primarni oddaljeni repozitorij. S tem ukažemo lokalnemu Gitu, kam naj v prihodnje pošilja kodo.

6. git push -u -f origin BRANCH_NA_KATEREGA_POTISKATE (npr. main)

- **Pomen:** Prenese lokalne commite in datoteke na oddaljeni strežnik GitHub v izbrano vejo (main).
- **Vloga zastavic:**
- -u (--set-upstream): Poveže lokalno vejo z oddaljeno vejo na GitHubu, tako da je v prihodnje dovolj vtipkati le git push ali git pull.
- -f (--force): Prisilno prenese spremembe in prepiše stanje na oddaljenem repozitoriju (uporabi se previdno, običajno pri prvem vzpostavljanju ali ponastavljanju repozitorija).

3. TEŽAVE

Pri praktični izvedbi vaje v okolju Windows PowerShell sem naletel na več konkretnih napak in izzivov pri uporabi ukazov Git:

1.Pozabljena pika pri ukazu git add:

- Težava: Vpisal sem samo ukaz git add brez pike ali imena datoteke, zaradi česar je Git javil opozorilo Nothing specified, nothing added. hint: Maybe you wanted to say 'git add .'?.
- Rešitev: Ponovil sem ukaz in pravilno vpisal git add ., kar je uspešno pripravilo datoteke na commit.

2.Napaka error: remote origin already exists:

- Težava: Ob ponovnem vnosu ukaza git remote add origin https://github.com/Anbatukham/VZI-praksa.git je Git javil napako, saj je bila povezava origin že ustvarjena.
- Rešitev: Obstoječo povezavo sem najprej odstranil z ukazom git remote remove origin ter jo nato znova pravilno dodal z git remote add origin ....

3.Napaka pri potisku zaradi manjkajočega imena veje (fatal: The current branch master has no upstream branch):

- Težava: Pri poskusu git push -u -f origin nisem navedel imena veje na koncu ukaza, zato me je Git opozoril, da veja master nima nastavljene sledilne veje na strežniku.
- Rešitev: Za uspešen potisk je bilo treba uporabiti celoten ukaz z opredeljeno vejo (git push --set-upstream origin master oz. git push -u origin master).

4. UGOTOVITVE

Na podlagi izvedenih korakov v terminalu PowerShell sem prišel do naslednjih ugotovitev:

1. **Uspešno izveden prvi commit:**

- Po popravku ukaza git add . sem z ukazom git commit -m "prvi commit - github" uspešno ustvaril prvi lokalni commit z ID-jem 06f908b.
- Iz izpisa je razvidno, da se je v sistemu Git potrdila 1 datoteka (VZI github.odt).

2. **Sledljivost napak v ukazni vrstici:**

- Terminal Git/PowerShell nudi zelo natančne namige (hints) in opozorila ob napakah. Iz posnetka je vidno, da Git sam predlaga ustrezno rešitev (npr. dodajanje pike pri git add . ali ukaz git push --set-upstream origin master).

3. **Pomen pravilne sintakse pri ukazih Git:**

- Ugotovil sem, da sta vrstni red parametrov ter navedba pravih imen vej (master ali main) ter oddaljenega strežnika (origin) ključna za uspešno sinhronizacijo lokalnih datotek z GitHubom.

5. ZAKLJUČEK

Z izvedbo te vaje in reševanjem začetnih izzivov v ukazni vrstici PowerShell sem spoznal osnove delovanja sistema Git in spletne platforme GitHub. Naučil sem se pravilne uporabe osnovnih ukazov za inicializacijo, pripravo, potrjevanje in pošiljanje datotek na oddaljeni strežnik. Kljub manjšim začetnim težavam in opozorilom v terminalu sem uspešno povezal lokalni repozitorij z GitHubom in prenesel svoje delo. Pridobljene izkušnje predstavljajo pomembno osnovo za učinkovito upravljanje datotek in kasnejše timsko sodelovanje pri predmetu.

6. VIRI :

1. Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. Dostopno na: https://git-scm.com/book/en/v2
2. GitHub Docs. (2026). *Getting started with GitHub and Git*. GitHub Documentation. Dostopno na: https://docs.github.com/en/get-started
3. Loeliger, J., & McCullough, M. (2012). *Version Control with Git: Powerful tools and techniques for collaborative software development*. O'Reilly Media.
4. Scott, T. (2020). *How Git Works: A Visual Guide to Version Control*. Pragmatic Programmers.